Visión computacional para imágenes y video · Grupo 10

Práctica 7 (parte A). Detector de esquinas de Harris

Comportamiento bajo cambios de iluminación, ángulo y escala

Integrantes:
Diego Falcón Costilla A01139580
Alejandro José Martínez Ubeda A01797775
Ana Bonavides Aguilar A01423281
José Andrés Orantes Guillén A01174130

Fecha: 1/11/2026

Entrega: Notebook ejecutado + PDF de resultados

7. Harris Edge & Corner Detection

Table of Contents

1. Libraries
2. Color image to Grayscale conversion
3. Spatial derivative calculation
4. Structure tensor setup
5. Harris response calculation
6. Find edges and corners using R

Importing Libraries

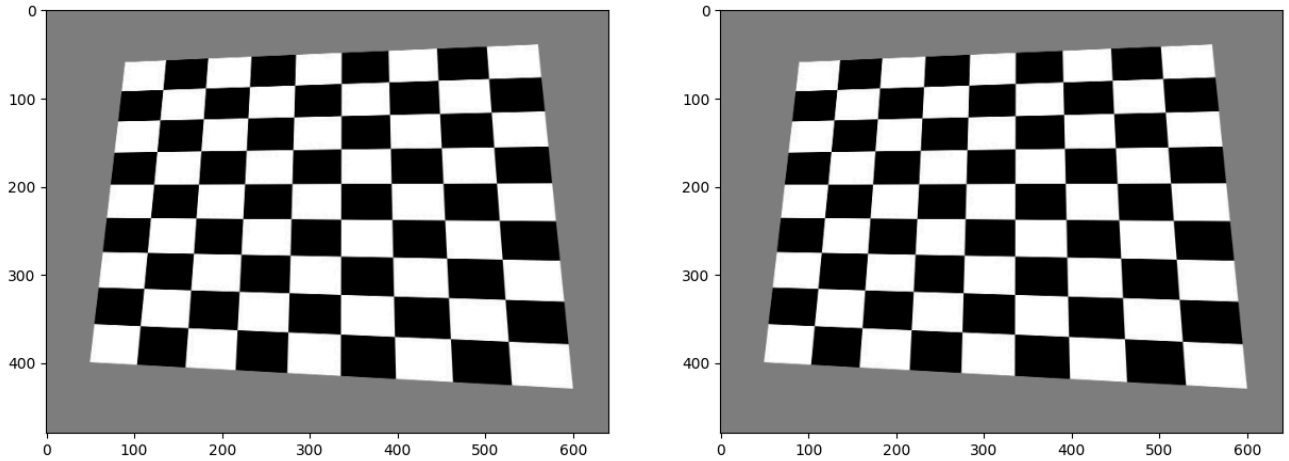
```
import cv2
import matplotlib.pyplot as plt
from scipy import signal as sig
import numpy as np
from scipy.ndimage import convolve
```

1. Color to Grayscale

```
img = cv2.imread('data/chessboard.jpg')
img_color = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(15, 8))
plt.subplot(1, 2, 1)
plt.imshow(img_color)
plt.subplot(1, 2, 2)
plt.imshow(img_gray, cmap="gray")
```

```
<matplotlib.image.AxesImage at 0x23d25cb4380>
```



2. Spatial derivative calculation

```
def gradient_x(imggray):
    ##Sobel operator kernels.
    kernel_x = np.array([[-1, 0, 1],[-2, 0, 2],[-1, 0, 1]])
    return sig.convolve2d(imggray, kernel_x, mode='same')

def gradient_y(imggray):
    kernel_y = np.array([[1, 2, 1], [0, 0, 0], [-1, -2, -1]])
    return sig.convolve2d(imggray, kernel_y, mode='same')

I_x = gradient_x(img_gray)
I_y = gradient_y(img_gray)
```

3. Structure tensor setup

```
def gaussian_kernel(size, sigma=1):
    size = int(size) // 2
    x, y = np.mgrid[-size:size+1, -size:size+1]
    normal = 1 / (2.0 * np.pi * sigma**2)
    g =  np.exp(-((x**2 + y**2) / (2.0*sigma**2))) * normal
    return g


Ixx = convolve(I_x**2, gaussian_kernel(3, 1))
Ixy = convolve(I_y*I_x, gaussian_kernel(3, 1))
Iyy = convolve(I_y**2, gaussian_kernel(3, 1))
```

4. Harris response calculation

```
k = 0.05

# determinant
detA = Ixx * Iyy - Ixy ** 2

# trace
traceA = Ixx + Iyy

harris_response = detA - k * traceA ** 2
```

```
img_gray.shape
```

```
(480, 640)
```

```
window_size = 3
offset = window_size//2
width, height = img_gray.shape

for y in range(offset, height-offset):
    for x in range(offset, width-offset):
        Sxx = np.sum(Ixx[y-offset:y+1+offset, x-offset:x+1+offset])
        Syy = np.sum(Iyy[y-offset:y+1+offset, x-offset:x+1+offset])
        Sxy = np.sum(Ixy[y-offset:y+1+offset, x-offset:x+1+offset])
```

```
#Find determinant and trace, use to get corner response
det = (Sxx * Syy) - (Sxy**2)
trace = Sxx + Syy
r = det - k*(trace**2)
```

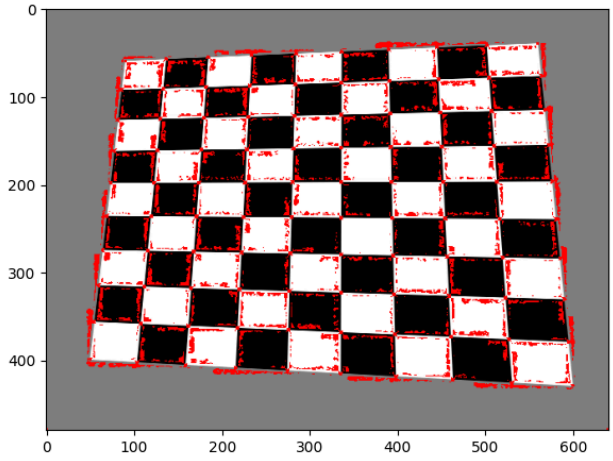
5. Find edges and corners using R

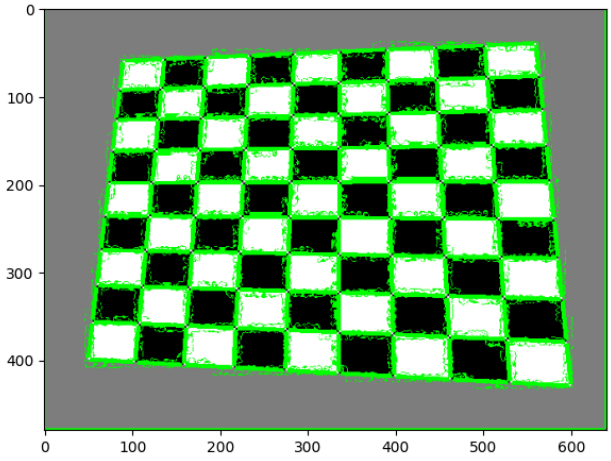
```
img_copy_for_corners = np.copy(img)
img_copy_for_edges = np.copy(img)

for rowindex, response in enumerate(harris_response):
    for colindex, r in enumerate(response):
        if r > 0:
            # this is a corner
            img_copy_for_corners[rowindex, colindex] = [255,0,0]
        elif r < 0:
            # this is an edge
            img_copy_for_edges[rowindex, colindex] = [0,255,0]
```

```
plt.figure(figsize=(15, 8))
plt.subplot(1, 2, 1)
plt.imshow(img_copy_for_corners, cmap="gray")
plt.subplot(1, 2, 2)
plt.imshow(img_copy_for_edges, cmap="gray")
```

```
<matplotlib.image.AxesImage at 0x23d25cb4950>
```





Experimentos: el detector de Harris bajo distintas condiciones

La actividad pide probar el detector con un objeto fotografiado bajo distinta iluminación y distintos ángulos. Como aquí no se dispone de fotos propias de cada condición, se parte de una fotografía real de un letrero (CC0, ver `LICENCIAS.md`) y se generan las demás vistas de forma controlada: los cambios de iluminación se simulan con brillo, gamma y ruido, y los de ángulo con la homografía exacta de un giro del plano del objeto, de la cámara o de la escala (`puntos.py`). Así se conoce la verdad y se puede medir la repetibilidad: de las esquinas detectadas en la imagen original, qué fracción reaparece (a ≤ 3 px) en la imagen transformada. Si se dispone de fotos propias, basta con reemplazar `data/letrero.jpg` y cargar cada toma como una condición adicional.

Contenido: 1. Condiciones · 2. Parámetros de Harris · 3. Hallazgos

```
import numpy as np, cv2, matplotlib.pyplot as plt
import puntos as P
plt.rcParams.update({"font.size": 9})

objeto = cv2.imread("data/letrero.jpg")
h, w = objeto.shape[:2]
gris0 = P.gris(objeto)
FOCO = 650
I = np.eye(3)
def rot(**kw): return P.homografia_rotacion(w, h, FOCO, **kw)

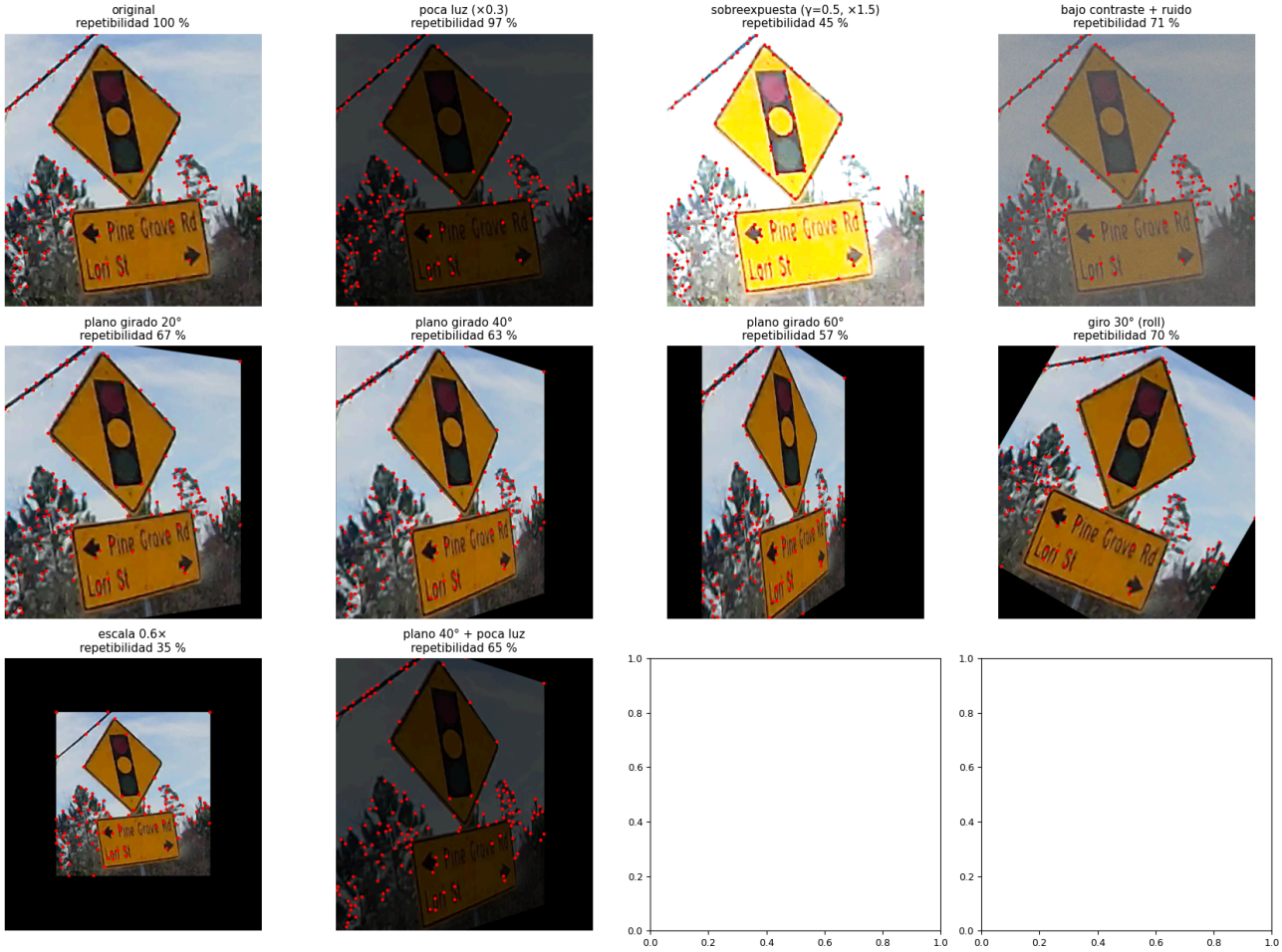
condiciones = {
    "original":                          (I, {}),
    "poca luz (×0.3)":                   (I, dict(brillo=0.3)),
    "sobreexpuesta (γ=0.5, ×1.5)":       (I, dict(gamma=0.5, brillo=1.5)),
    "bajo contraste + ruido":            (I, dict(brillo=0.45, offset=70, ruido=8)),
    "plano girado 20°":                  (P.homografia_plano(w, h, FOCO, 20), {}),
    "plano girado 40°":                  (P.homografia_plano(w, h, FOCO, 40), {}),
    "plano girado 60°":                  (P.homografia_plano(w, h, FOCO, 60), {}),
    "giro 30° (roll)":                   (rot(roll=30), {}),
    "escala 0.6×":                       (P.homografia_rotacion(w, h, FOCO, escala=0.6), {}),
    "plano 40° + poca luz":              (P.homografia_plano(w, h, FOCO, 40), dict(brillo=0.3)),
}
vistas = {n: P.vista(objeto, H, **kw) for n, (H, kw) in condiciones.items()}
print(f"Imagen del objeto: {w}×{h} px; esquinas de referencia: {len(P.detectar_harris(gris0, n=300))}")
```

```
Imagen del objeto: 470×500 px; esquinas de referencia: 241
```

1. Repetibilidad de Harris según la condición

Se toman las 300 esquinas más fuertes de la imagen original y de cada vista (Harris con ventana de 3 px, apertura de Sobel 3 y `k = 0.04`), y se mide qué fracción de las esquinas de la original que siguen dentro de la vista reaparecen a ≤ 3 px.

```
k0 = P.detectar_harris(gris0, n=300)
filas = []
fig, ejes = plt.subplots(3, 4, figsize=(17, 12.5))
for a, (nombre, (H, kw)) in zip(ejes.ravel(), condiciones.items()):
    v = vistas[nombre]; k1 = P.detectar_harris(P.gris(v), n=300)
    rep, visibles = P.repetibilidad(k0, k1, H, v.shape, 3.0)
    filas.append((nombre, len(k1), visibles, rep * 100))
    vis = v.copy()
    for kp in k1[:150]: cv2.circle(vis, (int(kp.pt[0]), int(kp.pt[1])), 3, (0, 0, 255), -1)
    a.imshow(cv2.cvtColor(vis, cv2.COLOR_BGR2RGB)); a.set_title(f"{nombre}\nrepetibilidad {rep*100:.0f} %"); a.axis
("off")
plt.tight_layout(); plt.show()
print(f"{'condición':32s} {'esquinas':>9s} {'visibles':>9s} {'repetibilidad %':>16s}")
for n, k, vs, r in filas: print(f"{n:32s} {k:9d} {vs:9d} {r:16.1f}")
```



condición	esquinas	visibles	repetibilidad %
original	241	241	100.0
poca luz (×0.3)	242	241	96.7
sobreexpuesta (γ=0.5, ×1.5)	251	241	44.8
bajo contraste + ruido	300	241	71.0
plano girado 20°	184	227	67.4
plano girado 40°	183	231	62.8
plano girado 60°	169	226	56.6
giro 30° (roll)	182	219	69.9
escala 0.6×	97	241	34.9
plano 40° + poca luz	189	231	64.9

2. Efecto de los parámetros del detector

La respuesta de Harris depende del tamaño de la ventana del tensor de estructura (`blockSize`) y del parámetro `k` . Se mide la repetibilidad en tres condiciones difíciles y el número de esquinas con respuesta mayor al 1 % del máximo.

```
dif = {"poca luz (×0.3)": (I, dict(brillo=0.3)), "plano girado 40°": (P.homografia_plano(w, h, FOCO, 40), {}), "gir
o 30° (roll)": (rot(roll=30), {}), "escala 0.6×": (P.homografia_rotacion(w, h, FOCO, escala=0.6), {})}
print(f"{'blockSize':>9s} {'k':>5s} {'esquinas orig.':>15s} " + " ".join(f"{n[:16]:>17s}" for n in dif))
for bloque in (2, 3, 5, 7):
    for kk in (0.04, 0.06):
        k0b = P.detectar_harris(gris0, n=300, bloque=bloque, k=kk)
        total = len(P.detectar_harris(gris0, n=100000, bloque=bloque, k=kk))
        fila = []
        for n, (H, kw) in dif.items():
            v = P.vista(objeto, H, **kw); k1 = P.detectar_harris(P.gris(v), n=300, bloque=bloque, k=kk)
            fila.append(P.repetibilidad(k0b, k1, H, v.shape, 3.0)[0] * 100)
        print(f"{bloque:9d} {kk:5.2f} {total:15d} " + " ".join(f"{x:16.1f}%" for x in fila))
```

blockSize	k	esquinas orig.	poca luz (×0.3)	plano girado 40°	giro 30° (roll)	escala 0.6×
2	0.04	212	94.3%	73.9%	63.2%	39.2%
2	0.06	210	98.1%	71.4%	60.3%	35.2%
3	0.04	241	96.7%	62.8%	69.9%	34.9%
3	0.06	233	95.7%	60.5%	65.6%	34.8%
5	0.04	230	98.3%	68.5%	74.6%	35.7%
5	0.06	224	98.7%	63.8%	70.3%	34.8%
7	0.04	201	98.0%	67.9%	77.5%	32.3%
7	0.06	191	97.4%	68.0%	77.4%	34.6%

3. Hallazgos

Iluminación.

- Harris resiste bien los cambios de brillo lineales: con la luz reducida a ×0.3 la repetibilidad es de 96.7 % (241 esquinas visibles y 242 detectadas). Un cambio de brillo multiplica los gradientes, pero no mueve el lugar donde la respuesta es máxima, y al quedarse con las 300 esquinas más fuertes el umbral se ajusta solo.
- Lo que sí lo daña es lo no lineal: con la imagen sobreexpuesta (γ=0.5, ×1.5) la repetibilidad cae a 44.8 %, porque las luces se saturan (el letrero es de un amarillo brillante) y las esquinas de las zonas saturadas desaparecen; con bajo contraste y ruido llega a 71.0 % y se detectan 300 esquinas (el límite), muchas de ellas espurias.

Ángulo.

- Al girar el plano del objeto (escorzo) la repetibilidad baja de forma gradual: 67.4 % (20°), 62.8 % (40°) y 56.6 % (60°). Una esquina deja de tener el mismo aspecto cuando la vecindad se deforma. Con poca luz añadida (40° + ×0.3) es de 64.9 %, es decir, la luz influye mucho menos que el ángulo.
- Con un giro de 30° en el plano de la imagen la repetibilidad es de 69.9 %. En teoría Harris es invariante a rotación (el determinante y la traza del tensor de estructura no cambian al girar), así que la pérdida se debe al remuestreo con interpolación, a que 22 de las 241 esquinas salen del encuadre y a que la posición se mide a ±3 px.
- Con una escala de 0.6× cae a 34.9 %: Harris no es invariante a la escala. Al reducir la imagen, una esquina se convierte en una curva suave dentro de la misma ventana. Esa es la razón de que SIFT trabaje en un espacio de escalas.

Parámetros. Aumentar `blockSize` mejora la repetibilidad frente al giro (63.2 % con 2, 69.9 % con 3, 74.6 % con 5 y 77.5 % con 7, con k = 0.04) y algo frente a la poca luz (94.3 % → 98.0 %), pero reduce el número de esquinas totales (212 → 201) y suaviza su localización. `k` (0.04 frente a 0.06) tiene un efecto pequeño y sin patrón consistente (diferencias de unos pocos puntos). Frente a la escala ningún ajuste ayuda (32–39 %).

Limitación. Las vistas son sintéticas a partir de una sola fotografía (sin paralaje, sombras ni reflejos reales). Con fotos propias del mismo objeto bajo el sol y en penumbra, y desde varios ángulos, se espera un comportamiento similar, con las diferencias que traigan las sombras y los reflejos.

Referencias

- Harris, C., & Stephens, M. (1988). A combined corner and edge detector. En *Proceedings of the 4th Alvey Vision Conference* (pp. 147–151). Alvey Vision Club.
- Schmid, C., Mohr, R., & Bauckhage, C. (2000). Evaluation of interest point detectors. *International Journal of Computer Vision, 37*(2), 151–172. https://doi.org/10.1023/A:1008199403446

- Gonzalez, R. C., & Woods, R. E. (2018). *Digital image processing* (4.ª ed.). Pearson.
- Bradski, G. (2000). The OpenCV library. *Dr. Dobb's Journal of Software Tools, 25*(11), 120–123.
- Imágenes: ver `LICENCIAS.md` (Wikimedia Commons, M. Rivera, CC0). El tablero `chessboard.jpg` se genera por código.

Visión computacional para imágenes y video · Grupo 10

Práctica 7 (parte B). Matching de puntos de interés

Harris, ORB y SIFT con variaciones de iluminación y orientación

Integrantes:
Diego Falcón Costilla A01139580
Alejandro José Martínez Ubeda A01797775
Ana Bonavides Aguilar A01423281
José Andrés Orantes Guillén A01174130

Fecha: 1/11/2026

Entrega: Notebook ejecutado + PDF de resultados

9. Image Matching

Table of Contents

1. Libraries
2. ORB Matching
3. SIFT Matching

Importing Libraries

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
```

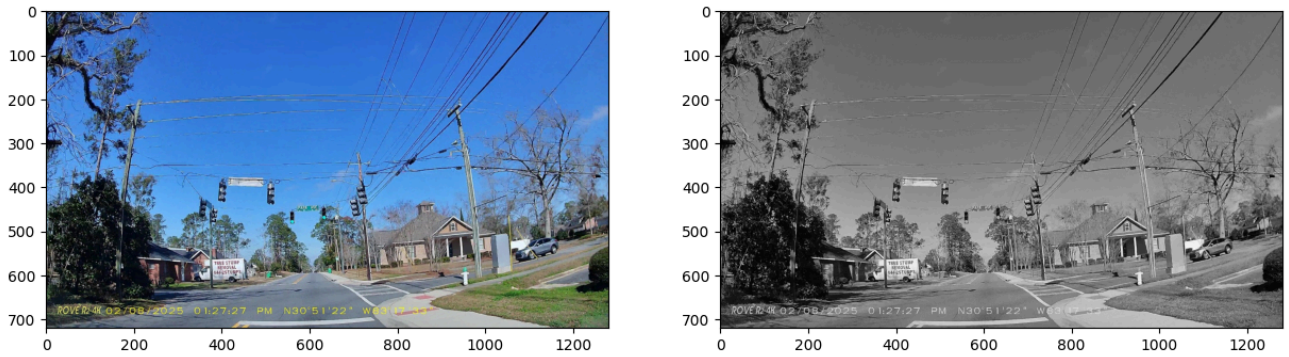
ORB (Oriented FAST and Rotated BRIEF)

- Developed at OpenCV labs by Ethan Rublee, Vincent Rabaud, Kurt Konolige, and Gary R. Bradski in 2011
- Efficient and viable alternative to SIFT and SURF (patented algorithms)
- ORB is free to use
- Feature detection
- ORB builds on FAST keypoint detector + BRIEF descriptor

```
#reading image
img = cv2.imread('data/escena_1.jpg')
img_color = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(15, 8))
plt.subplot(1, 2, 1)
plt.imshow(img_color)
plt.subplot(1, 2, 2)
plt.imshow(img_gray, cmap="gray")
```

```
<matplotlib.image.AxesImage at 0x1612fa13da0>
```



Create test image by adding Scale Invariance and Rotational Invariance

```
test_image = cv2.pyrDown(img_color)
test_image = cv2.pyrDown(test_image)
num_rows, num_cols = test_image.shape[:2]

rotation_matrix = cv2.getRotationMatrix2D((num_cols/2, num_rows/2), 30, 1)
test_image = cv2.warpAffine(test_image, rotation_matrix, (num_cols, num_rows))

test_gray = cv2.cvtColor(test_image, cv2.COLOR_RGB2GRAY)
```

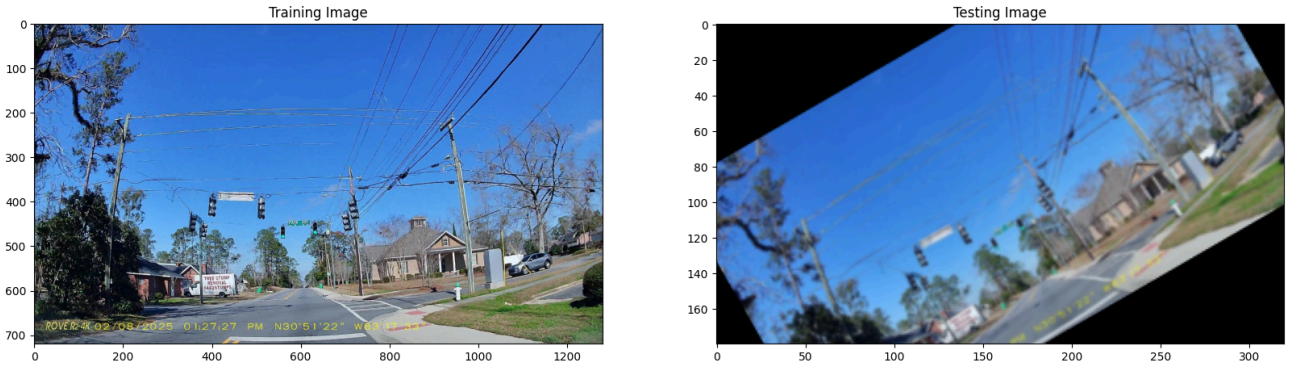
Display traning image and testing image

```
fx, plots = plt.subplots(1, 2, figsize=(20,10))

plots[0].set_title("Training Image")
plots[0].imshow(img_color)

plots[1].set_title("Testing Image")
plots[1].imshow(test_image)
```

```
<matplotlib.image.AxesImage at 0x16133819070>
```



ORB

```
orb = cv2.ORB_create()
```

```
train_keypoints, train_descriptor = orb.detectAndCompute(img_color, None)
test_keypoints, test_descriptor = orb.detectAndCompute(test_gray, None)

keypoints_without_size = np.copy(img_color)
keypoints_with_size = np.copy(img_color)

cv2.drawKeypoints(img_color, train_keypoints, keypoints_without_size, color = (0, 255, 0))

cv2.drawKeypoints(img_color, train_keypoints, keypoints_with_size, flags = cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOIN
TS)

# Display image with and without keypoints size
fx, plots = plt.subplots(1, 2, figsize=(20,10))

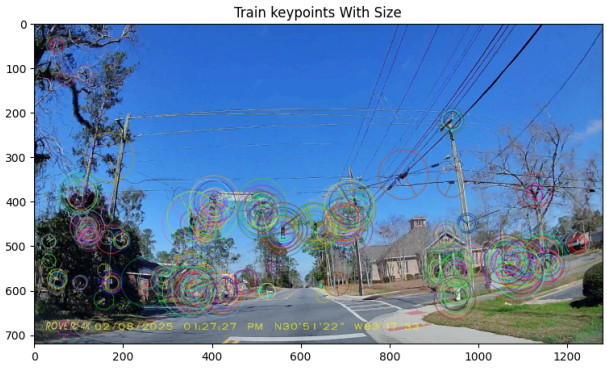
plots[0].set_title("Train keypoints With Size")
plots[0].imshow(keypoints_with_size, cmap='gray')

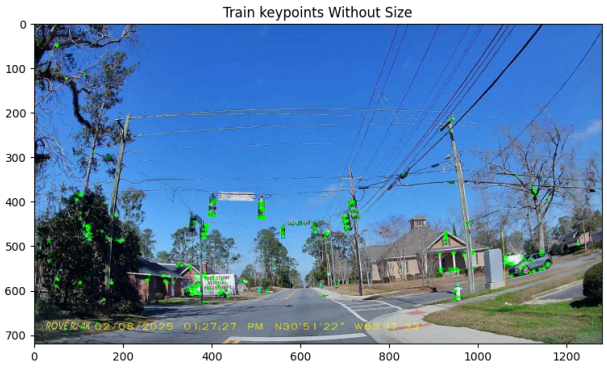
plots[1].set_title("Train keypoints Without Size")
plots[1].imshow(keypoints_without_size, cmap='gray')

# Print the number of keypoints detected in the training image
print("Number of Keypoints Detected In The Training Image: ", len(train_keypoints))

# Print the number of keypoints detected in the query image
print("Number of Keypoints Detected In The Query Image: ", len(test_keypoints))
```

```
Number of Keypoints Detected In The Training Image:  500
Number of Keypoints Detected In The Query Image:  352
```






```python
# Create a Brute Force Matcher object.
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck = True)

# Perform the matching between the ORB descriptors of the training image and the test image
matches = bf.match(train_descriptor, test_descriptor)

# The matches with shorter distance are the ones we want.
matches = sorted(matches, key = lambda x : x.distance)

result = cv2.drawMatches(img_color, train_keypoints, test_gray, test_keypoints, matches, test_gray, flags = 2)

# Display the best matching points
plt.rcParams['figure.figsize'] = [14.0, 7.0]
plt.title('Best Matching Points')
plt.imshow(result)
plt.show()

# Print total number of matching points between the training and query images
print("\nNumber of Matching Keypoints Between The Training and Query Images: ", len(matches))
```




```
Number of Matching Keypoints Between The Training and Query Images:  130
```

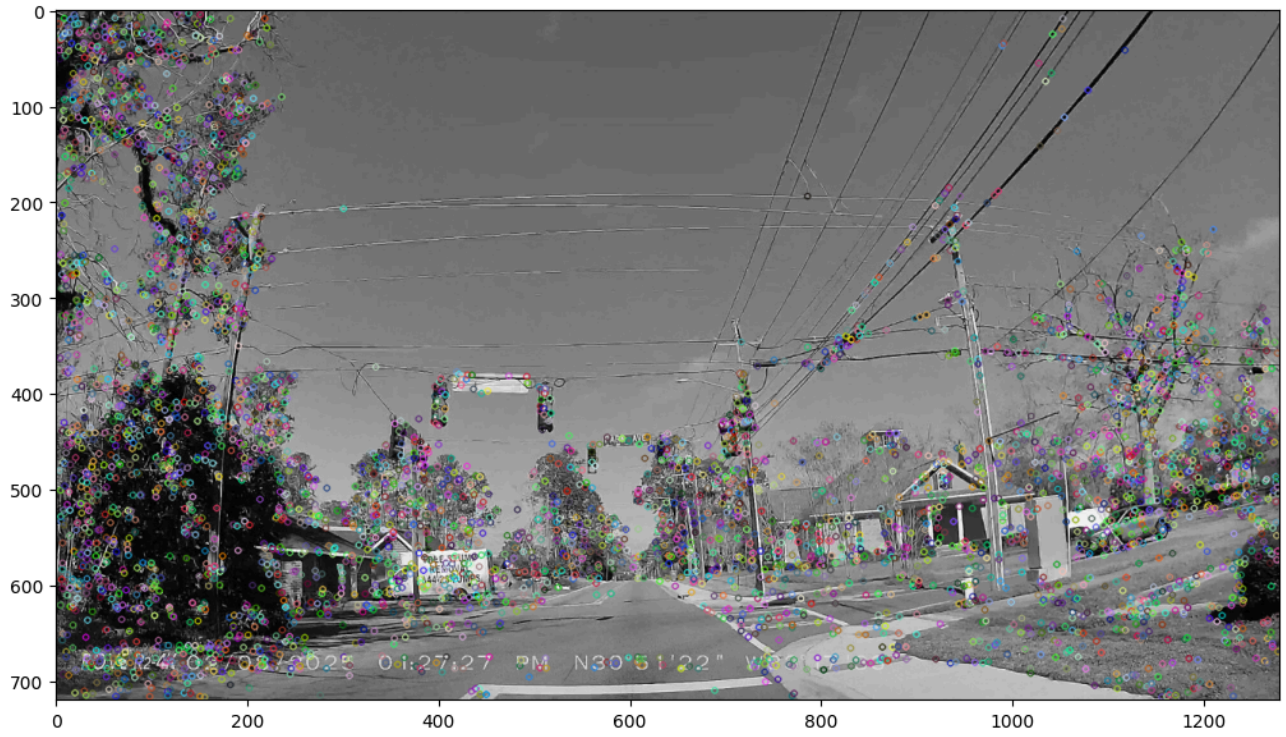
SIFT Matching (Scale Invariant Feature Transform)

```
img1 = cv2.imread('data/escena_1.jpg')
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)

#keypoints
sift = cv2.SIFT_create()
keypoints_1, descriptors_1 = sift.detectAndCompute(img1,None)

img_1 = cv2.drawKeypoints(gray1,keypoints_1,img1)
plt.imshow(img_1)
```

```
<matplotlib.image.AxesImage at 0x16133da50a0>
```



Matching different images

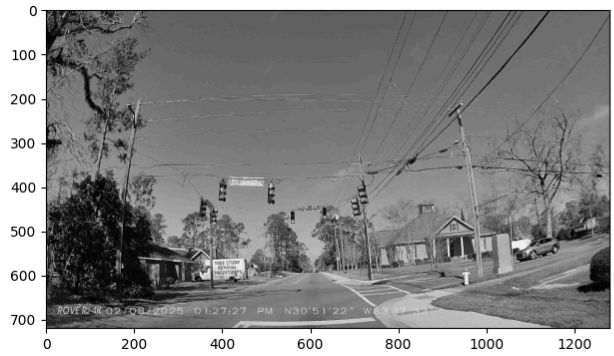
```
# read images
img1 = cv2.imread('data/escena_1.jpg')
img2 = cv2.imread('data/escena_2.png')

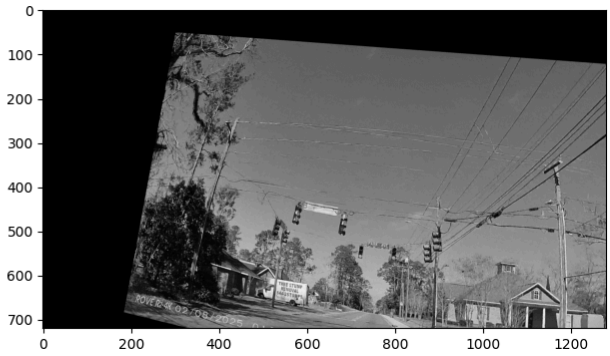
img1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
img2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

figure, ax = plt.subplots(1, 2, figsize=(16, 8))

ax[0].imshow(img1, cmap='gray')
ax[1].imshow(img2, cmap='gray')
```

```
<matplotlib.image.AxesImage at 0x161315f2c90>
```





Extracting Keypoints with SIFT

```
#sift
sift = cv2.SIFT_create()

keypoints_1, descriptors_1 = sift.detectAndCompute(img1,None)
keypoints_2, descriptors_2 = sift.detectAndCompute(img2,None)

len(keypoints_1), len(keypoints_2)
```

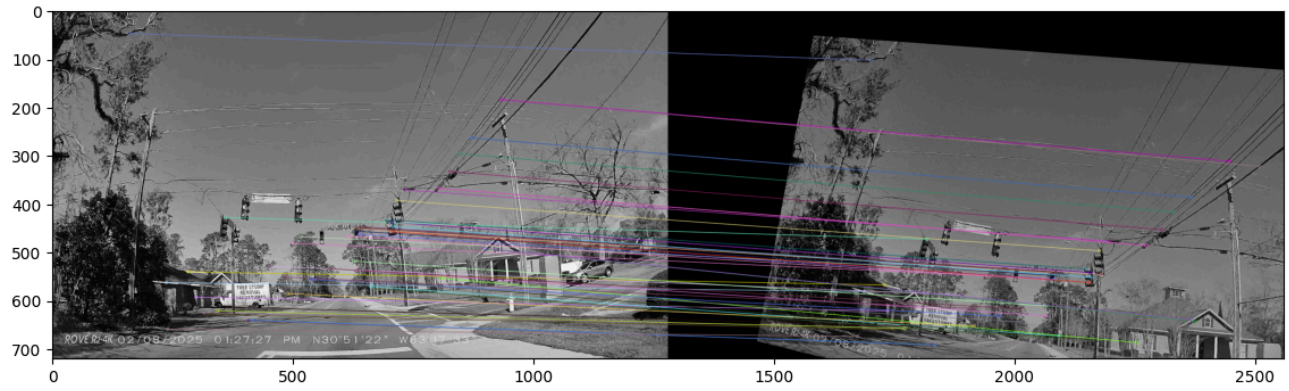
```
(4865, 3984)
```

Feature Matching

```
#feature matching
bf = cv2.BFMatcher(cv2.NORM_L1, crossCheck=True)

matches = bf.match(descriptors_1,descriptors_2)
matches = sorted(matches, key = lambda x:x.distance)

img3 = cv2.drawMatches(img1, keypoints_1, img2, keypoints_2, matches[:50], img2, flags=2)
plt.imshow(img3),plt.show()
```



```
(<matplotlib.image.AxesImage at 0x16133fb1e50>, None)
```

Experimentos: qué tan buenos son los descriptores para el matching

La actividad pide probar diferentes descriptores con variaciones de iluminación y orientación, y ver qué tan bien funciona el matching con Harris por sí solo o con otros como SIFT y ORB. Como las dos fotos de personas del notebook base (`elon_1` , `elon_2`) no se incluyeron, se usa una escena urbana real (CC0) y se generan vistas con transformación conocida (`puntos.py`), de modo que cada correspondencia se puede calificar como correcta o errónea.

Se comparan tres alternativas, todas con la misma prueba de razón de Lowe (0.8) y 1500 puntos como máximo:

Método	Detector	Descriptor	Distancia
Harris + parche	esquinas de Harris	parche 17×17 normalizado (media 0, norma 1)	L2
ORB	FAST orientado	BRIEF binario girado	Hamming
SIFT	diferencia de gaussianas (invariante a escala)	histogramas de gradientes (128 valores)	L2

«Harris por sí solo» no tiene descriptor propio: describe únicamente *dónde* hay esquinas; aquí se le da el descriptor más sencillo posible (el parche de píxeles) para emparejarlo.

```
import numpy as np, cv2, matplotlib.pyplot as plt
import puntos as P
plt.rcParams.update({"font.size": 9})

escena = cv2.imread("data/escena_1.jpg")
h, w = escena.shape[:2]
FOCO = 900
def rot(**kw): return P.homografia_rotacion(w, h, FOCO, **kw)
I = np.eye(3)
condiciones = {
    "sin cambio":                          (I, {}),
    "poca luz (×0.3)":                     (I, dict(brillo=0.3)),
    "sobreexpuesta (γ=0.5, ×1.5)":         (I, dict(gamma=0.5, brillo=1.5)),
    "bajo contraste + ruido":              (I, dict(brillo=0.45, offset=70, ruido=8)),
    "giro 30° (roll)":                     (rot(roll=30), {}),
    "giro 90° (roll)":                     (rot(roll=90), {}),
    "perspectiva yaw 25°":                 (rot(yaw=25), {}),
    "escala 0.6×":                         (P.homografia_rotacion(w, h, FOCO, escala=0.6), {}),
    "giro 30° + poca luz":                 (rot(roll=30), dict(brillo=0.3)),
}
metodos = ["harris", "orb", "sift"]
res = {}
for n, (H, kw) in condiciones.items():
    v = P.vista(escena, H, **kw)
    for m in metodos:
        res[(n, m)] = P.evaluar(escena, v, H, m, n=1500, ratio=0.8)[0]
print("CORRESPONDENCIAS CORRECTAS (de las totales), precisión y matches consistentes con la homografía estimada (RA
NSAC)\n")
print(f"{'condición':30s} " + " ".join(f"{m:>25s}" for m in metodos))
for n in condiciones:
    print(f"{n:30s} " + " ".join(f"{res[(n, m)]['correctos']:5d}/{res[(n, m)]['matches']:<5d} {res[(n, m)]['precisi
on']*100:5.1f}% RS{res[(n, m)]['inliers_ransac']:<5d}" for m in metodos))
```

```
CORRESPONDENCIAS CORRECTAS (de las totales), precisión y matches consistentes con la homografía estimada (RANSAC)

condición                                harris                      orb                     sift
sin cambio                  1357/1357  100.0% RS1357   1500/1500  100.0% RS1500   1500/1500  100.0% RS1500
poca luz (×0.3)             1293/1296   99.8% RS1293   1285/1309   98.2% RS1285   1215/1226   99.1% RS1215
sobreexpuesta (γ=0.5, ×1.5)  323/327    98.8% RS323     184/219    84.0% RS182     262/284    92.3% RS262
bajo contraste + ruido       931/939    99.1% RS931    1254/1271   98.7% RS1254    878/896    98.0% RS878
giro 30° (roll)                0/5       0.0% RS4      786/827    95.0% RS787     837/856    97.8% RS837
giro 90° (roll)                0/3       0.0% RS0      567/624    90.9% RS582     651/662    98.3% RS651
perspectiva yaw 25°          219/227    96.5% RS219     522/555    94.1% RS519     546/574    95.1% RS546
escala 0.6×                   11/17     64.7% RS11      519/534    97.2% RS518     243/256    94.9% RS243
giro 30° + poca luz            0/5       0.0% RS4      701/742    94.5% RS692     669/688    97.2% RS669
```

```
print("REPETIBILIDAD DEL DETECTOR (%) y TIEMPO (ms, detección en las dos imágenes)\n")
print(f"{'condición':30s} " + " ".join(f"{m:>18s}" for m in metodos))
for n in condiciones:
    print(f"{n:30s} " + " ".join(f"{res[(n, m)]['repetibilidad']*100:8.1f}% {res[(n, m)]['t_deteccion_ms']:6.0f}ms"
for m in metodos))
print("\nError medio de la homografía estimada (px, en las 4 esquinas; NaN si RANSAC no encontró modelo)\n")
print(f"{'condición':30s} " + " ".join(f"{m:>10s}" for m in metodos))
for n in condiciones: print(f"{n:30s} " + " ".join(f"{res[(n, m)]['error_H_px']:10.2f}" for m in metodos))
```

REPETIBILIDAD DEL DETECTOR (%) y TIEMPO (ms, detección en las dos imágenes)

condición	harris		orb		sift	
sin cambio	100.0%	57ms	100.0%	23ms	100.0%	108ms
poca luz (×0.3)	96.4%	48ms	94.2%	17ms	83.6%	103ms
sobreexpuesta (γ=0.5, ×1.5)	45.9%	47ms	49.2%	24ms	34.1%	106ms
bajo contraste + ruido	81.1%	47ms	95.9%	19ms	68.7%	118ms
giro 30° (roll)	73.5%	92ms	85.3%	21ms	82.0%	120ms
giro 90° (roll)	99.6%	43ms	85.9%	27ms	95.8%	116ms
perspectiva yaw 25°	64.2%	45ms	91.7%	30ms	67.0%	108ms
escala 0.6×	32.7%	52ms	89.5%	19ms	35.2%	111ms
giro 30° + poca luz	73.1%	47ms	81.8%	17ms	65.4%	128ms

Error medio de la homografía estimada (px, en las 4 esquinas; NaN si RANSAC no encontró modelo)

condición	harris	orb	sift
sin cambio	0.00	0.00	0.00
poca luz (×0.3)	0.01	0.05	0.01
sobreexpuesta (γ=0.5, ×1.5)	0.10	0.44	0.21
bajo contraste + ruido	0.01	0.08	0.04
giro 30° (roll)	805.10	1.95	0.18
giro 90° (roll)	nan	2.11	0.51
perspectiva yaw 25°	1.02	7.59	0.39
escala 0.6×	1.39	2.19	0.17
giro 30° + poca luz	805.10	3.05	0.15

```
# Visualización: giro de 30° con poca luz (el caso más difícil de la tabla). Verde = correspondencia correcta, rojo = errónea.
H, kw = condiciones["giro 30° + poca luz"]
v = P.vista(escena, H, **kw)
fig, ejes = plt.subplots(3, 1, figsize=(15, 14))
for a, m in zip(ejes, metodos):
    r, (k1, k2, ms) = P.evaluar(escena, v, H, m, n=1500, ratio=0.8)
    a.imshow(cv2.cvtColor(P.dibujar_matches(escena, k1, v, k2, ms, H, max_n=60), cv2.COLOR_BGR2RGB))
    a.set_title(f"{m.upper()}: {r['correctos']} correctas de {r['matches']} ({r['precision']*100:.0f} %); se muestran las 60 mejores"); a.axis("off")
plt.tight_layout(); plt.show()
```

HARRIS: 0 correctas de 5 (0 %); se muestran las 60 mejores



ORB: 701 correctas de 742 (94 %); se muestran las 60 mejores



SIFT: 669 correctas de 688 (97 %); se muestran las 60 mejores



Hallazgos

Cambios de iluminación (sin cambiar la geometría).

- Con poca luz (×0.3) los tres funcionan casi igual: Harris 1293/1296 correctas (99.8 %), ORB 1285/1309 (98.2 %) y SIFT 1215/1226 (99.1 %). Con bajo contraste y ruido, todos superan el 98 %.
- La sobreexposición con saturación es el caso más difícil de los cambios fotométricos: Harris conserva la precisión (98.8 %) pero con muy pocas correspondencias (323), SIFT baja a 92.3 % (262/284) y ORB a 84.0 % (184/219). Los descriptores binarios de ORB comparan pares de píxeles y se confunden cuando las zonas saturadas pierden textura.

Cambios de orientación y escala.

- Harris por sí solo no sirve con giros: con 30° solo produce 5 correspondencias y ninguna es correcta (0/5), y con 90° solo 3 (0/3). El descriptor de parche compara los píxeles tal como están y no es invariante a la rotación; el detector encuentra las mismas esquinas (repetibilidad de 73.5 % con 30° y 99.6 % con 90°, donde el giro exacto no interpola), pero no hay cómo reconocerlas. Con la escala de 0.6× solo logra 11/17.
- ORB y SIFT funcionan con giros y escala: con 30° ORB 786/827 (95.0 %) y SIFT 837/856 (97.8 %); con 90° ORB 567/624 (90.9 %) y SIFT 651/662 (98.3 %); con escala 0.6× ORB 519/534 (97.2 %) y SIFT

243/256 (94.9 %). Combinar giro y poca luz (30° + ×0.3) no cambia esa conclusión (ORB 94.5 %, SIFT 97.2 %).
- Con una perspectiva moderada (yaw 25°) hasta el parche de Harris sirve (219/227, 96.5 %), pero con menos correspondencias que ORB (522) o SIFT (546).

Precisión geométrica y tiempo.

- SIFT es el más preciso: el error de la homografía estimada (RANSAC) no supera 0.51 px en ninguna condición. ORB llega a errores de entre 0.05 y 7.6 px (el peor caso, con la perspectiva de 25°). Con solo 4 o 5 correspondencias, la homografía de Harris no tiene sentido (errores de cientos de píxeles).
- En tiempo de detección (dos imágenes de 1280×720, un solo equipo y varias ejecuciones), ORB fue el más rápido (unos 20–40 ms), Harris quedó en un punto intermedio (45–110 ms) y SIFT fue el más lento (105–220 ms), varias veces más que ORB (entre 3 y 8 veces según la condición). Los valores exactos cambian de una ejecución a otra, pero el orden se mantuvo.

Conclusiones. (1) Harris detecta *dónde* hay puntos estables; para emparejar hace falta un descriptor, y con uno ingenuo (píxeles del parche) no soporta rotación ni escala. (2) SIFT ofrece la mejor precisión y la mayor robustez, a cambio de tiempo de cómputo; ORB es una alternativa mucho más rápida con precisión ligeramente menor, salvo con saturación. (3) Ninguno resolvió sin errores la saturación de luces. (4) Los resultados dependen de la escena y de la prueba de razón (0.8); son de una sola escena con vistas sintéticas y deberían complementarse con fotos reales propias.

Referencias

- Harris, C., & Stephens, M. (1988). A combined corner and edge detector. En *Proceedings of the 4th Alvey Vision Conference* (pp. 147–151). Alvey Vision Club.
- Lowe, D. G. (2004). Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision, 60*(2), 91–110. https://doi.org/10.1023/B:VISI.0000029664.99615.94
- Rublee, E., Rabaud, V., Konolige, K., & Bradski, G. (2011). ORB: An efficient alternative to SIFT or SURF. En *2011 International Conference on Computer Vision* (pp. 2564–2571). IEEE. https://doi.org/10.1109/ICCV.2011.6126544
- Fischler, M. A., & Bolles, R. C. (1981). Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM, 24*(6), 381–395. https://doi.org/10.1145/358669.358692
- Mikolajczyk, K., & Schmid, C. (2005). A performance evaluation of local descriptors. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 27*(10), 1615–1630. https://doi.org/10.1109/TPAMI.2005.188
- OpenCV. (s. f.). *Feature detection and description*. https://docs.opencv.org/4.x/db/d27/tutorial_py_table_of_contents_feature2d.html
- Imágenes: ver `LICENCIAS.md` (Wikimedia Commons, M. Rivera, CC0).

Declaración y uso de IA

Para la elaboración de este trabajo se utilizó Claude Opus, de Anthropic, como herramienta de apoyo en el desarrollo y la revisión del código, así como en la redacción del documento final. Su uso fue complementario y no sustituyó el criterio académico de los autores. El análisis, la interpretación de los resultados y la formulación de las conclusiones corresponden a los autores, quienes asumen la responsabilidad íntegra sobre la revisión, la exactitud y la integridad académica del contenido final.